

Step 2: Setting up memcached

Download the memcached source from <http://memcached.org/> and install it. Start *memcached* on the local system on two ports, using the following commands:

```
memcached -p 11212 &
memcached &
```

I now have two *memcached* instances, one running on port 11212 and the second on the default port (i.e., 11211).

```
$ telnet 0 11211
Trying 0.0.0.0...
Connected to 0.
Escape character is '^J'.

$ telnet 0 11212
Trying 0.0.0.0...
Connected to 0.
Escape character is '^J'.
```

What next? Get the *memcached_demo.py* script, for which you can use the following command:

```
git clone git@github.com:sukujgrg/mysql_memcached.git
```

What this script does

This script contacts the local MySQL database and executes two SQL fetch queries (below). On the first execution of the script, it will print the output of the query, which is directly fetched from the database, and will also store that data in a locally running memcache instance. From the next execution onwards, this script will directly fetch data from the memcache. The SQL queries to fetch data in this script are:

```
select * from ips_class where c_class like '192.168.1.25%';
select count(*) from ips_class;
```

On first execution of the script:

```
$ python memcached_demo.py
Fetched from DB (Execution time = 17.4954309464 seconds)
Total IPs in the DB
= 33554431
Fetched from DB (Execution time = 18.8784821033 seconds)
All IPs in DB which is matching to the regex 192.168.1.25*
= 192.168.1.25 192.168.1.250 192.168.1.251 192.168.1.252
192.168.1.253 192.168.1.254 192.168.1.255
```

On second execution of the script:

```
$ python memcached_demo.py
Cached Result (Execution time = 0.000431060791016 seconds)
Total no. of IPs in the DB
= 33554431
Cached Result (Execution time = 0.00189614295959 seconds)
All IPs in DB which is matching to the regex 192.168.1.25*
= 192.168.1.25 192.168.1.250 192.168.1.251 192.168.1.252
192.168.1.253 192.168.1.254 192.168.1.255
```

You can clearly see the difference between the 'Execution time' in both cases. Very impressive, right?

Of course, there are many options, including invalidating or expiring the data, in memcached. For more on the topic, delve deeper into the *References*. 

References

- [0] <http://www.ibm.com/developerworks/xml/library/os-memcached/index.html>
- [1] <http://www.adayinthelifeof.nl/2011/02/06/memcache-internals/>
- [2] <http://dev.mysql.com/doc/mysql-ha-scalability/en/ha-memcached-interfaces-python.html>
- [3] <http://blog.echolibre.com/2009/11/memcache-and-python-getting-started/>

By: Suku

The author is an open source fan. Feel free to respond to this article and direct your feedback to sukujgrg@gmail.com.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

Your favourite Magazine on Open Source is now on the Web, too.

LinuxForU.com

Follow us on Twitter@LinuxForYou